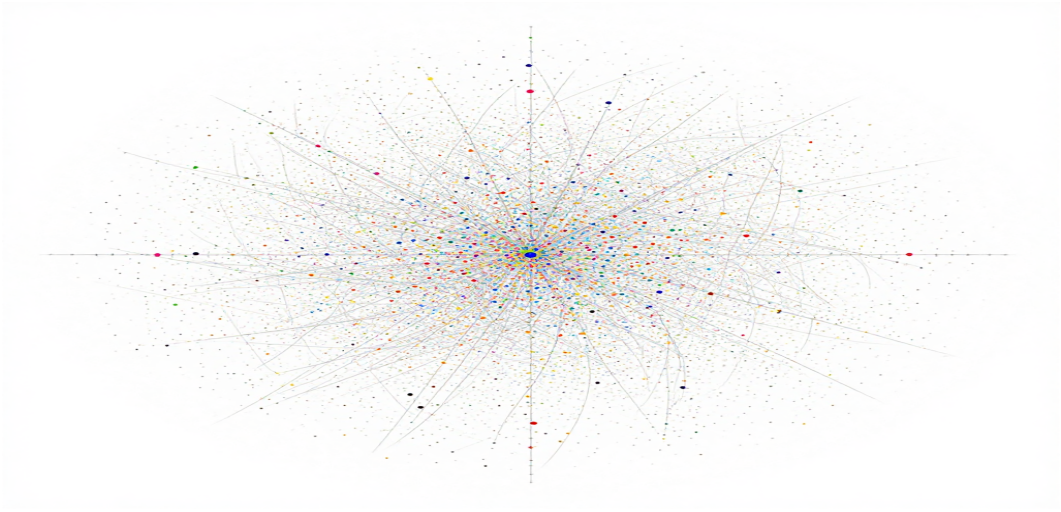
CHALMERS
UNIVERSITY OF TECHNOLOGY | UNIVERSITY OF GOTHENBURG



Applying and Evaluating a Non-Functional Requirements Management Framework for a GraphRAG-Based Documentation Chatbot: An Industrial Case Study

Bachelor of Science Thesis in Software Engineering and Management

EMMA OLMÅS

ERIK LIDBOM

FREDRIK NILSSON

Department of Computer Science and Engineering
UNIVERSITY OF GOTHENBURG
CHALMERS UNIVERSITY OF TECHNOLOGY
Gothenburg, Sweden 2026

Applying and Evaluating a Non-Functional Requirements Management Framework for a GraphRAG-Based Documentation Chatbot

Supervisor: ANTONIA WELZEL
Examiner: GREGORY GAY

University of Gothenburg
Chalmers University of Technology
Department of Computer Science and Engineering
SE-412 96 Göteborg
Sweden
Telephone + 46 (0)31-772 1000

[Cover: AI generated picture of a graph]

Department of Computer Science and Engineering
University of Gothenburg
Chalmers University of Technology
Gothenburg, Sweden 2026

Applying and Evaluating a Non-Functional Requirements Management Framework for a GraphRAG-Based Documentation Chatbot: An Industrial Case Study

Emma Olmås
Computer Science and Engineering
University of Gothenburg
Gothenburg, Sweden
gusolmem@student.gu.se

Erik Lidbom
Computer Science and Engineering
University of Gothenburg
Gothenburg, Sweden
guslidboer@student.gu.se

Fredrik Nilsson
Computer Science and Engineering
University of Gothenburg
Gothenburg, Sweden
gusnilfral@student.gu.se

***Abstract*—Conversational chatbots based on Large Language Models and Machine Learning work as good support for developers in navigating technical documentation, but designing and developing such systems introduces challenges in managing non-functional requirements due to the non-deterministic nature of Machine Learning systems. This case study applies and evaluates an existing five-step framework for managing non-functional requirements in Machine Learning based systems. The framework is applied during the development of a Graph Retrieval-Augmented Generation-based documentation chatbot for a node-based data analysis system. Data is collected through semi-structured interviews, Focus Group sessions, requirement artifacts produced throughout the framework application and by quantitative evaluation of the chatbot. The findings will provide insight into how structured non-functional requirements management can support the development of Graph Retrieval-Augmented Generation systems in industrial environments and highlight practical considerations for managing non functional requirements in machine learning-driven developer support tools.**

I. Introduction

Developing Machine Learning (ML) systems involves significant challenges in managing non-functional requirements (NFRs). Traditional requirements engineering struggles with ML's non-determinism, data dependencies, and evolving needs [1]. Quality concerns like performance, accuracy, and maintainability often conflict across data, models, and deployment components, and these trade-offs prove difficult to identify, prioritize, scope, and measure systematically [2]. Empirical work with practitioners confirms this difficulty: in a survey of 278 industry practitioners working on ML-based systems [3], more than 80 percent rated most ML engineering activities as moderately to extremely difficult, with existing approaches often no longer applicable. This gap between established engineering practice and the demands of ML-based systems motivates the need for structured approaches to manage NFRs systematically.

Graph Retrieval-Augmented Generation (GraphRAG) documentation chatbots exemplify these ML challenges. GraphRAG constructs entity-relationship graphs from data input to enable enhanced retrieval and grounded generation. However, building and maintaining these systems creates heavy preprocessing demands, ongoing update costs, slower query responses, and scaling problems as documentation grows [4]. To tackle the challenges of NFRs in ML-systems, Habibullah et al. [1] propose an iterative framework that guides practitioners through five steps: identifying/prioritizing NFR types, defining/scoping them contextually, analyzing trade-offs, creating measurement catalogues, and specifying concrete instances. This structured approach moves from broad concerns to actionable requirements, making it well-suited for industrial ML projects like GraphRAG chatbots. Since the framework has not yet been broadly validated, further application in different industrial settings is needed to understand its strengths and limitations in practice. To address the research gap, this thesis applies and evaluates the framework during the development of a GraphRAG-based chatbot for a node-based data analysis system, using interviews, focus groups, and artifacts to manage NFRs in a real industrial context.

The study bridges GraphRAG complexities to practical requirements engineering via the following research questions:

RQ1: *How can the quality requirements management framework be applied to manage non-functional requirements in a GraphRAG-based documentation chatbot?*

This question focuses on how the framework is instantiated in the case, including how NFRs are identified, prioritized, scoped, measured, and specified during the development of the chatbot.

RQ2: *To what extent does the framework support effective quality requirements management in this GraphRAG-based system?*

This question focuses on the perceived usefulness of the framework in practice, including whether it makes the management of quality requirements clearer, more systematic, and easier to discuss and document.

II. Related Work

A. GraphRAG and Documentation-Grounded Chatbots

Documentation-grounded chatbots are increasingly relevant in software-intensive environments where users need support in understanding concepts, navigating workflows, and solving problems using technical documentation [5]. In such settings, Retrieval-Augmented Generation (RAG) allows Large Language Models (LLMs) to access external knowledge beyond their training data [6]. The RAG architecture GraphRAG transforms data input into entity-relationship graphs for even more advanced retrieval [4]. Edge et al. [4] introduce GraphRAG indexing with local entity-level and global community-based search for query-focused summarization, outperforming baselines on complex queries while noting preprocessing time costs.

Recent surveys on GraphRAG examine workflows, evaluation benchmarks and domain-specific implementations across different application contexts. For example, Peng et al. [7] provides a comprehensive survey of GraphRAG analyzing workflows and performance comparisons, and the survey by Han et al. [8] focuses on framework designs and relational trade-offs. At the same time, the graph construction and maintenance process introduces additional complexity compared to simpler retrieval pipelines, and research on enterprise use of RAG systems shows that practical concerns such as latency, scalability, privacy, hallucination control, and domain adaptation become important in real-world deployment [8]. This means that the development of a GraphRAG-based chatbot is not only a question of answer quality, but also of how relevant quality requirements are identified and managed during development.

Developing a GraphRAG-based documentation chatbot therefore involves managing multiple competing quality requirements systematically. Teams must identify which requirements matter most, define how each applies across system components, and make explicit trade-off decisions. This complexity suggests that systematic approaches to requirements management are necessary when developing GraphRAG systems in industrial settings.

B. Management of Quality Requirements

Managing design choices in AI chatbots is complicated by ML's unique nature, where systems introduce hidden technical debt through complex data dependencies, non-determinism, and maintenance issues that are not immediately obvious [9]. Research shows that poor design often causes performance failures, emphasizing the need for early assessment of NFRs like latency and resource utilization [10].

The study by Habibullah et al. [2] reports that participants regard NFRs as playing a vital role in the success of ML-enabled software systems. However, the research also states that requirements engineering is the most challenging activity for ML-enabled software development - there is a lack of knowledge due to the lack of documentation, methods, and benchmarks to define and measure NFRs for ML systems [2]. Additionally, the research shows varied answers for the scope of NFR definition in ML-enabled systems where most participants focused on the ML model itself and not necessarily on the whole software and even less focus was placed on NFRs over data [2]. Another challenge when managing NFRs in ML-based systems is that new requirements appear. For example, retrainability is a new non-functional requirement for ML-systems that is not applicable to traditional software since it covers the identification of when, how, and which data to retrain [2].

A recent systematic mapping study covering 126 papers on requirements engineering for AI-based systems reinforces these observations [11]. The study identifies the lack of suitable RE methodologies for ML systems as one of the most common challenges in the field, alongside the difficulty of managing NFRs whose definitions and priorities shift in the ML context. Habiba et al. [11] point to NFR management for ML as an open research direction, indicating that the issues raised in earlier practitioner studies remain unresolved at a field-wide level.

One example of how NFRs change in the ML context is explainability. Köhl et al. [12] argue that explainability should be treated as an NFR on its own, not just as a technical feature of the model. They also point out that what counts as a good explanation depends on who is asking, what they want to understand, and the situation in which the explanation is given. This makes RE and NFR management harder, because there is no single clear definition of what the requirement actually means. For instance, different stakeholders may expect different things from the same chatbot answer, which makes it difficult to specify and measure explainability in a consistent way.

C. Quality Trade-offs in Industrial Practice

Industrial case studies show that quality trade-offs between performance, cost, and accuracy are real concerns in ML-enabled systems [13]. A recent systematic review [14] of RAG and LLMs in enterprise knowledge management and document automation shows that real-world use involves more than model accuracy on benchmarks. It stresses practical challenges such as latency, scalability, privacy, security, hallucination control, and domain adaptation, and highlights the gap between laboratory studies and real-world deployment [14]. The review remains broad and does not provide detailed empirical evidence on how RAG developer support systems behave in enterprise software environments. This gap motivates the need for more systematic requirements engineering support in enterprise RAG settings. In particular, it suggests that practitioners need help identifying, prioritizing, and balancing NFRs when developing such systems in real organizational contexts.

D. A Framework for Managing Quality Requirements in ML-Based Systems

The research by Habibullah et al. [1] describes how software systems have become increasingly dependent on ML and since

it is difficult to identify functional requirements over ML components, the importance of satisfied NFRs is very high. Research shows that it is difficult to define different types of NFRs both in terms of where in the system the NFRs are important but also including prioritization, lack of knowledge of the practitioners but also the lack of effective tools to help define and measure NFRs for ML systems [1].

Since ML systems contain interaction code and non-code components, different NFRs may be of different importance based on which part of the system is analysed and the definition of the NFRs may also shift based on the different parts of the system [1].

Based on these findings, Habibullah et al. [1] propose a framework for quality requirements management in ML based systems. The framework is based on the following five steps [1]:

Step 1 - Select and prioritize NFR types. During this step, practitioners should select a subset of NFR types that might be of importance to their system. Then, the practitioners should rank the selected NFR types as *'Very Important', 'Important'* or *'Not Important'*. This approach ensures the efficiency of research allocation and helps the practitioners to make informed design decisions.

Step 2 - Define NFRs and their scope over the system. A definition catalogue should be created, containing the prioritized NFR types and their general definitions, to establish a shared understanding between practitioners. Step 2 also includes scoping the selected NFRs over the system, and this part is important since the definition, importance, and measurement of NFRs may differ between the different components of a system.

Step 3 - Identify trade-offs. Trade-offs might appear between the different NFR types and practitioners need to identify these and find a balance between them. This part is useful since, if a conflict appears, practitioners will discuss and provide a rationale to further know which NFR to prioritize and why.

Step 4 - Create a measurement catalogue. This step includes creating a measurement catalogue containing the selected NFRs together with measurements and equations for each NFR, which will be used to assess the satisfaction of each NFR, at each scope, in the system. This approach provides a systematic method that helps practitioners define measurements, identify targets, and detect potential measurement challenges in an early stage of development.

Step 5 - Document specific NFR instances. Since each NFR type might have multiple specific NFRs, it is important to document all of them to achieve a shared understanding between the practitioners and to ensure that all NFRs are satisfied. The NFR instances catalogue contain each of the NFR instances, with details such as measurement, measurement target, trade-offs.

The framework steps are intended to be iterative, which means that teams may move back and forth between steps if needed, and the purpose of it is to help practitioners move from broad quality concerns to more concrete and documented requirements.

In detail, the framework intend to help practitioners define and prioritize NFR types, identify NFR type scope, balance NFR type trade-offs, specify an NFR measurement catalogue and specify an NFR instance template.

E. Research Gap Summary

Previous research shows that GraphRAG and other ML-based systems involve important quality trade-offs [7] [8]. At the same time, previous work shows that NFRs in ML-based systems are difficult to identify, define, scope, and measure systematically [2]. These findings indicate that there is a need to explore how to identify and balance NFRs in ML-based systems.

While Habibullah et al. [1] propose a framework to manage NFRs in ML systems, there is still limited research on how it can be applied and evaluated in the development of a GraphRAG-based documentation chatbot in an industrial context. This study addresses that gap by instantiating the framework in such a case and examining its usefulness for managing NFRs in practice.

III. Methodology

A. Overview and Research Strategy

During this research, we perform a single industrial case study. A case study is suitable because the research examines a real software engineering problem in its natural setting and aims to understand how a requirements framework can be applied in practice. Case studies are appropriate when the phenomenon is closely connected to its context and when multiple sources of evidence are used, such as interviews and project artifacts [15] [16].

The study is mainly exploratory and descriptive. It investigates how an existing framework for managing NFRs in ML-based systems can be applied to a GraphRAG system. It documents how the framework is used to identify, prioritize, scope, measure and specify requirements for the chatbot.

The framework was applied step by step during the study which aligns with the procedure by Habibullah et al. [1]. The research included five steps divided into four sessions - a first round of interviews, three focus groups where Focus Group 3 also included evaluative questions about the framework. Each step of this research corresponds to one or more of the steps in the framework, see Figure 1.

The collected material was then qualitatively analyzed using thematic analysis based on the steps by Braun & Clarke [17]. The prioritized requirements defined during the framework application were used as guidelines when implementing the chatbot, and the measurement targets were then compared with the finished system through a quantitative analysis to determine whether the targets were achieved.

The results of these activities contribute to answering the two research questions. RQ1 is answered by the step-by-step application of the framework across the interviews and focus groups, and by the requirement artifacts produced at each step.

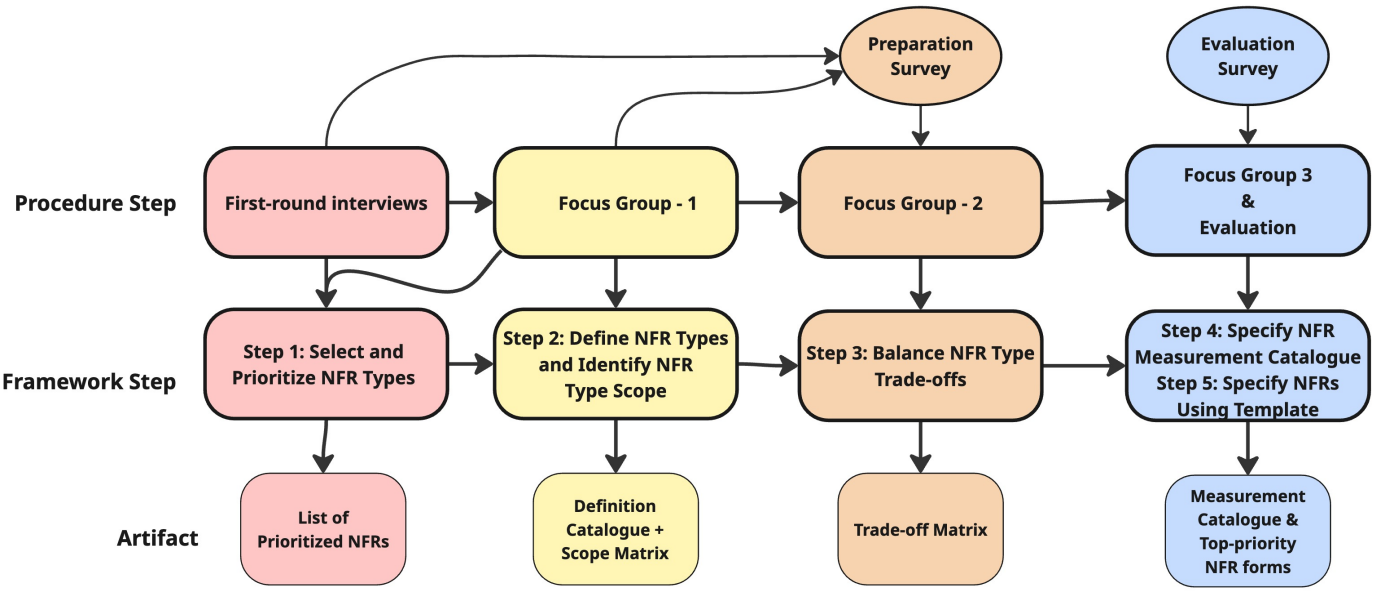


Fig. 1. Overview of the steps of the research, with a presentation of how each step is connected to a step of the framework produced by Habibullah et al. [1]. Beside these steps, the implementation of the chatbot was made with support of the created artifacts.

The artifacts document how NFRs were identified, prioritized, scoped, traded off, measured, and specified.

RQ2 is answered through the thematic analysis, where participants discussed the steps while applying them and reflected on the framework. Additionally, RQ2 is answered through the quantitative evaluation, which tests whether the finished system met the measurement targets the framework helped define.

B. Research Context

The study is in collaboration with a company developing a node-based visual data analysis and manipulation system. The chatbot is intended as an interactive support tool grounded in the company's documentation, helping users complete workflows, understand concepts, and troubleshoot issues. The company context provides real documentation, realistic user needs, and operational constraints from an actual industrial setting.

The documentation consists of user manuals, tutorials, API references and release notes, which form the knowledge base for the chatbot. The domain is appropriate for the study because documentation-heavy systems place clear demands on NFRs such as groundedness (answers derived and anchored in the documentation), performance, and maintainability. It therefore provides a relevant industrial setting for applying and evaluating the framework for managing NFRs in ML-systems on a GraphRAG-based system.

C. Data Collection

Data was collected through semi-structured interviews and focus groups. The interviews used pre-defined open-ended questions with the possibility to explore participants' responses through follow-up questions. The focus groups were used to create, refine and discuss the requirement artifacts produced during the framework application together with the participants. This approach is suitable for a case study since it enables detailed insights into participants' expectations, quality concerns, and requirements related to the chatbot, while also capturing how the framework was applied in practice [20].

The notes and audio recordings from the sessions captured decisions, disagreements and changes to the artifacts and were used as support during the analysis. The artifacts created were used during development of the chatbot, and then data about the defined NFRs was collected through testing. After each data collection session, the notes were cleaned, the audio recordings transcribed, and all data were anonymized prior to analysis.

The first round of interviews and Focus Group 1 were not audio-recorded, one researcher asked the questions while the other two researchers took notes. At the time, we did not consider audio recording necessary, as we believed that shared note-taking between the researchers would be sufficient to capture the key points from the sessions. When coding this material for the thematic analysis, we found that notes alone sometimes lost the exact wording and reasoning behind participants' statements. Focus Group 2 and Focus Group 3 were therefore audio-recorded and later transcribed, while also noted down, to ensure the accuracy of the data used in the analysis.

1) Subjects: The participant selection followed a purposive sampling strategy. The subjects in the study are software developers, product owners and data scientists involved in the data analysis system and the chatbot. Participants were selected based on the following criteria: direct working knowledge of the data analysis system, its documentation, the chatbot, and a role relevant to the subject. These criteria ensured

that participants could meaningfully contribute to discussions about quality concerns and NFRs in the chatbot's context, while their differing backgrounds, fields of work, and roles brought diverse perspectives to the sessions. Table I shows the participants in our case study, including which sessions each participated in.

TABLE I
PARTICIPANTS DURING DATA COLLECTION

	Participant 1	**Participant 2**	**Participant 3**
Field of Work	Software Engineering	Software Engineering, Data Science & AI	Data Science & Machine Learning
Role	Product Owner & Developer	Lead Software Developer	Data Scientist & Project Lead
Experience (years)	13 (1.5 as Product Owner)	2	3
ML Experience (years)	1 (during studies, limited at work)	2 (during studies and bachelor thesis)	3
NFR Experience (years)	1 (indirect at work, not explicitly defined)	1 (during studies, limited at work)	3 (considered during work, not formally defined)
First-Round Interview	X	X	
Focus Group 1	X	X	
Preparation Survey			X
Focus Group 2	X	X	X
Focus Group 3	X	X	
Focus Group 3 Survey			X

Table of information about the participants.

2) Procedures: The framework by Habibullah et al. [1] was used as the main structure for the requirements work in this study. To capture how the framework was applied in practice, data was collected in four stages, see Figure 1. This approach was used to capture expectations before development and opinions after development, as well as the process of applying the framework during the development of the chatbot. This is important for answering the research questions where RQ1 focuses on how the framework is applied, and RQ2 focus on observations and participant feedback gathered to assess the framework's perceived usefulness.

First Round of Interviews: Each interview took 90 minutes where one researcher asked the questions while the other two took notes. The purpose was to collect information about the system context, the engineers that participated and to identify important NFRs for the chatbot. The participants were asked about their role, years of experience with ML and NFRs, and overall experience of their field of work. The participants were then given a set of 10 NFRs, with questions regarding the requirement's definition and importance. The selection of NFRs was based on the researchers experience and perception of the most common and important NFRs for chatbots, and the participants were asked to think of additional requirements that could be of importance to the system. The interview guide was reviewed with our supervisor before use and adjusted from her feedback. The complete interview guide is provided in Appendix A.

Focus Group 1: The focus group took 90 minutes and this time there were only two researchers present, one researcher lead the focus group and the other took notes. During the focus group, the participants first reviewed and refined the initial list of prioritized NFRs, created during their interviews, and then reviewed the proposed definition catalogue created by the researchers and discussed whether each definition reflected the intended meaning of the corresponding NFR. Finally, participants were presented with an empty scope table, whose columns the researchers had set to represent the parts of a GraphRAG-based system based on their own knowledge of the architecture. The participants filled in this table together by discussing, for each NFR, which parts of the system it could meaningfully be defined over. The complete Focus Group 1 guide is provided in Appendix B.

Preparation Survey:

A third participant was available for Focus Group 2 and received a Preparation Survey to align with the prior work carried out by Participants 1 and 2. The survey contained a short version of the first-round interview- and Focus Group 1 guide, including background questions about the participant, and a prioritization task asking which NFRs they found most relevant for the chatbot. The survey also included the results from Focus Group 1. The completed survey was returned to the researchers prior to Focus Group 2. The survey can be found in Appendix C.

Focus Group 2: Focus Group 2 took 70 minutes and was audio recorded. One researcher led the session, and the others took notes.

This session involved discussion about trade-offs between the prioritized NFRs where the participants discussed which NFRs are most likely to conflict in the chatbot, and which NFR should be prioritized if such a conflict appears. The complete focus group guide is provided in Appendix D.

Focus Group 3: Focus Group 3 took 75 minutes and was audio recorded. Two researchers were present, one researcher led the session and the other took notes. The researchers proposed a measurement catalogue containing example measurements and targets for each prioritized NFR. The participants got to discuss and refine the measurements and targets to suit the chatbot context and system, and removed unnecessary measurements. As a final evaluation step, the participants discussed their experiences with the framework, including whether it made the management of quality requirements clearer, more systematic, and easier to discuss and document. The Focus Group 3 guide is provided in Appendix E.

Evaluation Survey: Participant 3 was unfortunately unable to participate in Focus Group 3. A survey containing the evaluation questions from Focus Group 3 was sent, and can be found in Appendix E.

Data Collection of Chatbot Requirements: The chatbot was developed alongside the progression of the framework application. As the study progressed through the steps, the artifacts produced in Steps 4 and 5 gave the team more concrete guidance on where to focus implementation effort, translating the prioritized NFR types and their measurements targets into further development decisions. The data collected here are measurements of the running chatbot's behaviour against those targets. Whether the system met those targets also speaks to RQ2, since a framework that produced clear,

actionable measurements is on that supported effective NFR management in practice.

Of the 25 measurements defined in Step 4, only five could be computed within this study due to the absence of test users and time constraints: Error Rate (Reliability), Average Response Time and Maximum Response Time (Performance), and Answer Correctness and Groundedness (Accuracy). The remaining measurements required either real users interacting with the system or infrastructure that was outside the scope of this study.

The data collection was performed on the chatbot prototype running on a local machine. The chatbot uses Microsoft's GraphRAG framework with AI model gpt-4.1 for generation, gpt-4.1-mini for graph indexing, and text-embedding-3-large for embeddings. All model calls went to the OpenAI API. We did not override the model sampling parameters in the GraphRAG configuration, so generation used the framework's default sampling. The chatbot used GraphRAG's local search, which does not set a temperature, so generation calls used the OpenAI default temperature of 1.0. Documents were chunked into 1800-token segments with a 50-token overlap, and graph extraction used the entity types node, library, data type, parameter, api, concept, and file format.

The test set had 31 prompts across five categories: 8 short, factual questions about a specific concept, 15 how-to questions about performing a task, 2 broader questions about the system as a whole, 2 large workflow questions covering several parts of the documentation, and 4 off-topic questions to test how the chatbot handles questions outside its domain, and these the chatbot should refuse. Of the 31 questions, 27 were written by Participant 1, from their knowledge of the documentation, and frequently asked questions from users. The 27 questions were reviewed by Participant 2 to check that the set covered the system and reflected realistic user questions. The remaining four off-topic questions was written by the researchers. The test set was performed under the same procedure three times, where each question was run once per test. The full test set can be found in Appendix G.

Each question was submitted to the chatbot through Microsoft's BenchmarkQED, a benchmarking tool for RAG systems. For each prompt, BenchmarkQED recorded the answer, response time, and whether the call succeeded, feeding the Error Rate and Response Time measurements. For the Accuracy measurements, BenchmarkQED also runs an LLM-as-a-judge (gpt-4.1) against each question's assertions: one for correctness (the answer states the documented facts accurately) and one for groundedness (the answer is anchored in the documentation), using gpt-4.1 with a pass threshold of 0.5 and four trials per question-assertion pair. The judge was given a system prompt instructing it to act as an impartial evaluator: parse the assertion to understand what condition must be met, examine the full answer for evidence that satisfies or contradicts it, and return a binary score of 1 (satisfied) or 0 (not satisfied) together with a reasoning citing specific evidence from the answer. For each prompt and run, the question, answer, time elapsed, success flag, and all BenchmarkQED scores were recorded. No manual validation of the judge's verdicts was performed, which we discuss in the Threats to Validity section.

D. Data Analysis

1) Qualitative Analysis: The interview data was qualitatively analyzed using thematic analysis based on the steps described by Braun & Clarke [17]. The analysis had an open and data-driven exploratory approach, focused on identifying recurring themes related to important NFRs, requirements trade-offs, and perceptions of the usefulness of the framework. The codes were not defined in advance, but emerged from the data during reading. The only structure imposed beforehand came from the first-round interview guide, which organized questions around the ten NFR categories. This provided a high-level frame but did not predetermine the codes, which were drawn from participants' own words and reasoning.

The analysis followed an iterative process and was carried out in conjunction with data collection. After each interview and focus group, the material was read through and broken down into codes that captured the meaning of individual sentences or statements. Each code was recorded together with the original sentence, the source from which it came, and the NFR to which it was related, if applicable. Codes were then grouped based on what they had in common, first sub-themes, and then into broader, main themes, see Appendix F. Each researcher took responsibility for coding a subset of the data, while the other two reviewed the resulting codes. The initial themes were collectively developed and the themes were revised as new data was added. The codes, sub-themes and main themes was also reviewed and approved by the university Supervisor.

Some codes did not cluster into themes. These included background information about the participants or the system, and single mentions that did not recur in other sources. These codes were kept in the dataset but are not represented in the themes.

2) Quantitative Analysis: The measurements were computed directly from the BenchmarkQED output, using equations defined during Step 4, and compared to the targets from the matching NFR instances of Step 5.

$$ER = \frac{N_{\text{fail}}}{N_{\text{total}}} \times 100\%,$$

Error Rate (ER) was computed with N_{total} and N_{fail} where N_{total} is the total number of questions submitted in a run (31), and N_{fail} is the number of requests in that run that timed out, raised an exception, or returned an empty response. The factor of 100 converts the proportion to a percentage. The target from RE002 is $\leq 5\%$. A request counted as a failure if it timed out, raised an exception, or returned no content. A wrong answer was not counted as a failure, but instead answer quality is captured by Groundedness. We report the raw count alongside the percentage, since with $N = 31$ a single failure shifts the rate by about 3 percentage points.

$$ART = (1/N_{\text{success}}) \times \sum t_i,$$

Average Response Time (ART) was computed by all requests, divided by the number of successful requests N_{success}, where N_{success} is the number of successful requests in the run ($N_{\text{total}} - N_{\text{fail}}$), and t_i is the response time in seconds of the i-th successful request. Failed requests were excluded because their elapsed time reflects the timeout limit, not the chatbot's actual behavior. The target from PE001 is ≤ 5 seconds. We also report the median (p50) and 95th percentile (p95), since the median shows typical behavior and p95 shows the slow tail without depending on a single extreme value.

$$MRT = \max_i t_i$$

Maximum Response Time (MRT) was computed over the successful requests, where t_i is the response time in seconds of the i-th successful request, and the maximum is taken over all successful requests in the run. Failed requests were again excluded, since failures contribute to Error Rate instead. The target from PE002 is ≤ 10 seconds. With one submission per question and three runs, the data doesn't support meaningful hypothesis testing, rather descriptive and focused on whether targets were met.

$$AC = (N_{\text{correct}}/N_{\text{eval}}) \times 100\%,$$

Answer Correctness (AC) was computed with $N_{\text{eval}} = 31$ and N_{correct} where N_{eval} is the total number of questions evaluated (31), and N_{correct} is the number of questions whose correctness assertion passed the judge. The target from AC001 is $\geq 90\%$.

$$G = (N_{\text{grounded}}/N_{\text{eval}}) \times 100\%,$$

Groundedness (G) was computed with $N_{\text{eval}} = 31$ and N_{grounded} where N_{eval} is the total number of questions evaluated (31), and N_{grounded} is the number of questions whose groundedness assertions both passed the judge. The target from AC002 is $\geq 90\%$.

E. Threats to Validity

Construct validity: One threat to construct validity is that participants may understand concepts such as usefulness, quality, or NFRs in different ways. To reduce this risk, we prepared structured guides that introduced and explained the relevant concepts while leaving room for participants to ask clarifying questions during the sessions, see Appendices A-E.

Another threat is the fact that there were different amounts of participants during the sessions due to people not being available to participate every session. To mitigate this, the Preparation Survey was sent out before Focus Group 2 to align the new participant with the discussions and artifacts from first-round interviews and Focus Group 1. The same participant was also given an Evaluation Survey after Focus Group 3 to capture their final reflections on the framework.

A further threat is that all participants are affiliated with the same company and share assumptions about the documentation, the product, and the user base. This could narrow the range of NFRs and trade-offs that surface during the framework application, which would affect how completely we can judge the framework's behavior even within this case. We addressed this through triangulation of interviews, focus groups, and framework artifacts. The analysis explicitly compares participant views.

Another threat concerns the accuracy measurements. Answer Correctness and Groundedness were scored entirely by an LLM-as-a-judge with no manual validation of its verdicts. Since the judge itself is an LLM, it may misinterpret a question or answer, meaning the scores can't be fully trusted as ground truth. This risk is compounded by the fact that the judge used the same model as the chatbot's generator (gpt-4.1), making the evaluation partly a self-assessment that may be biased toward accepting the system's own answer. A judge from a different model family, or a human judge, would potentially give a stricter result.

Internal validity: A threat to internal validity is that our own interpretations may affect the thematic analysis of the interview material. To reduce this risk, all authors reviewed the coding and discussed the results together.

The small sample of two to three purposively selected participants also threatens the validity. These developers and product owner might emphasize technical NFRs like accuracy and performance. They may under-represent perspectives from the actual end-users who might prioritize NFRs such as explainability or usability.

A related threat is the participants' limited experience with NFRs. As shown in Table I, their NFR experience ranged from one to three years and was more informal rather than explicit Requirement Engineering. This may have shaped how they prioritized and defined the NFR types, and it also shapes their assessment of the framework, since part of what they were judging was their own difficulty with concepts they had not worked with formally before. This same limitation is part of what RQ2 examines, however, since how much prior expertise the framework assumes is one of the things this study set out to learn.

Another threat to internal validity arises from researcher bias in the framework application. Although the participants got the chance to add, remove and refine the NFRs during the sessions, the researchers handled the implementation and artifact creation. Our development choices could unconsciously influence these, such as narrowing performance scopes to fit timelines. However, because all participants get to review and refine the proposed material, the risk is being reduced.

The quantitative evaluation also has limits. The chatbot is operational, but it is a Minimum Viable Product (MVP), and there is still a lot of refining left to do. Within the time available, we could not iterate on the system enough to reach all measurement targets, and we could not compute the measurements that need user testing or longer operational data. The quantitative analysis is therefore limited to the subset of measurements we could compute within the scope of the thesis. The remaining measurements are still documented in the catalogue, but were not evaluated in this study.

The quantitative evaluation also has no baseline. We did

not build a comparable chatbot without the framework, so the measurements alone can't attribute the system's quality to the framework rather than to ordinary development work. The quantitative evidence therefore speaks to consistency between the documented decisions and the system's behaviour, rather than to an improvement in quality. We rely on the thematic analysis for evidence of the framework's contribution, where participants reflected directly on how the framework shaped their decisions and what it added over an unstructured approach.

External validity: Regarding external validity, the study only focuses on one company and one chatbot case. This means that the result cannot automatically be generalized to all GraphRAG systems. However, the study can still provide useful insights for similar industrial cases.

Another threat is again the different numbers of participants during the sessions. With this, the findings reflect the views of a different subset of perspectives at different stages of the framework application. This makes it harder to argue that the results would generalize to settings where a more stable group is involved throughout, and means that some artifacts were shaped by fewer voices than others.

Reliability: A risk regarding reliability is that another research team might interpret the material differently. To reduce this risk, the interview process, the framework application, and the analysis steps were documented as clearly as possible, and the interview and focus group guides are included in Appendices A-E.

Non-recorded sessions also threaten reliability, since notes alone might miss details or reflect selective memory. The first-round interviews and Focus Group 1 were not audio-recorded, and this initial experience led us to recognize the importance of audio recordings for preserving details. Focus Group 2 and 3 were therefore audio-recorded and transcribed, which mitigates this threat for the later sessions. For the non-recorded sessions, we summarized notes immediately after each session, shared summaries with the participants so they could confirm or correct what we had written. All authors reviewed the anonymized notes before coding.

IV. Results

This section describes the findings of the study in three parts. First, we present the requirement artifacts produced across the interviews and focus groups together with which step of the framework it fulfills. Secondly, we report the quantitative results of the evaluation of the chatbot prototype against the subset of measurements that could be computed within the scope of this study. Finally, we present the themes that emerged from the thematic analysis of the qualitative material.

A. First-round Interviews

In both interviews, the participants described the system as a visual data-analysis system where users often struggle to get started, understand the documentation, and to know which nodes or workflow to use. In both cases, the chatbot was mainly described as a support tool that should help bridge the gap between documentation and actual work. Participant 2 stated that the chatbot is expected *"To help the users to find the right sections of the documentation, to refer to the documentation, give a brief and simple description of which components to use when."* Another statement was that: *"It shall help the user to configure the nodes correctly."* (P1). The interview questions and answers worked as support for the initial artifacts that was produced during the first-round interviews:

1) Step 1 - Initial Prioritization: This step involved for the participants to set the priority level (Very Important, Important or Not Important) on the proposed NFRs, which included: Reliability, Accuracy, Usability, Explainability, Performance, Security, Scalability, Maintainability, Portability and Safety.

The participants joined on Usability and Security priority level to be Very Important. Additionally, both participants found Reliability, Explainability and Performance as Important. They also joined on Portability as Not Important. Participants gave concrete reasons for several of these ratings. Reliability was treated as Important rather than Very Important because the chatbot was seen as "*more of a feature than something critical, nothing else depends on it*" (P1), with existing email support providing a fallback when it is unavailable. Scalability was rated Not Important by P2 and Important by P1, with P2 reasoning that "*if we get 10,000 new users, we will also get enough revenue to handle it*". The participants differed on Maintainability and Safety: Participant 1 found both Very Important, arguing that "*it is of bigger importance that the chatbot is easy to maintain and update rather than it is super complex and perfect*", while Participant 2 rated Maintainability as Important and Safety as Not Important, reasoning that heavy effort would be required to misuse the chatbot and that it can't take harmful actions directly anyways. The initial prioritizations given by the participants are presented in Table II.

TABLE II
Initial step 1 prioritization of NFRs

Requirement	Very Important	Important	Not Important
Reliability		1-2	
Usability	1-2		
Explainability		1-2	
Accuracy	2	1	
Performance		1-2	
Security	1-2		
Scalability		1	2
Maintainability	1	2	
Portability			1-2
Safety	1		2

Prioritization given by Participant 1 and Participant 2 during the interviews. An "1-2" in the same cell indicates that both participants had the same priority level for that NFR.

B. Artifact Creation 1

1) Step 2 - Creation of the Initial NFR Type Definition Catalogue: This step included the creation of the Initial NFR Type Definition Catalogue. The catalogue was created by the

researchers, based on material from the interviews. Several of the definitions reflect concerns that the participants raised consistently across both interviews. The framings quoted below are the participants' own words rather than wording introduced by the researchers, and they carried through to the refined definitions in Table V.

Accuracy was framed as *"documentation-grounded correctness*", (P2), rather than general correctness, anchoring the chatbot's expected behavior in the company's own documentation. The participant also described accurate answers as those that stay *"close to the ground truth and do not drift off."*, (P2). **Usability** was described more in practical, simple terms with the chatbot expected to give *"clear answers that are easy to translate into how to use it in the system"* (P1), keeping it easy to get started rather than becoming another interface to learn. **Security** was framed briefly as the prevention of unauthorized exposure of sensitive user, company, or conversation data, with one participant stating *"company data must not leak to competitors"* (P2). **Safety** was defined as avoiding harmful, inappropriate, or misleading behavior while remaining within the intended domain, tying the safety concern to the chatbot staying inside its scope.

The Initial NFR Type Definition Catalogue is presented in Table III.

TABLE III
INITIAL NFR TYPE DEFINITION CATALOGUE

NFR	Initial definition
Reliability	The chatbot operates consistently during normal use and handles failures in a clear and recoverable way.
Usability	The chatbot supports simple, clear and practical responses for users.
Explainability	The chatbot provides understandable reasons for its recommendations and can point to relevant documentation.
Accuracy	The chatbot gives correct and documentation-grounded answers.
Performance	The chatbot provides responses within a time range that supports interactive use.
Security	The chatbot prevents unauthorized exposure of sensitive user, company, workflow, and conversation data.
Scalability	The chatbot maintains acceptable operation as documentation, users and usage volume increase.
Maintainability	The chatbot can be updated, debugged, and improved efficiently over time.
Portability	The chatbot backend can be deployed in different technical environments without major redesign.
Safety	The chatbot avoids harmful, inappropriate, or misleading behaviour and remains within its intended domain.

Table with initial definitions created by the researchers that was used as support during Focus Group 1.

C. Focus Group 1

The session produced three refined artifacts: a prioritization list of ten NFR types, an updated definition catalogue, and a scoping table over the GraphRAG architecture.

1) Step 1 - Refined Prioritization: The participants were presented with the initial prioritization list, showing where the participants were joined in selected NFRs priority level, and where they differed. The participants already agreed on Usability and Security to be Very Important, and through discussion, Accuracy and Maintainability was set as Very Important as well. Safety was set as Important, between the two original ratings of Very Important and Not Important. Scalability was settled as Not Important and the other NFR types remained at their initial priority level. The joined, final list of prioritization NFRs are available in Table IV.

TABLE IV
STEP 1 - PRIORITIZATION LIST OF NFRS

Requirement	Very Important	Important	Not Important
Reliability		X	
Usability	X		
Explainability		X	
Accuracy	X		
Performance		X	
Security	X		
Scalability			X
Maintainability	X		
Portability			X
Safety		X	

Table with the list of prioritized NFRs defined during Focus Group 1.

2) Step 2 - Refined Definitions: The participant reviewed the Initial NFR Type Definition Catalogue, and three definitions were refined: Accuracy, Performance, and Portability. **Accuracy** was extended to refer explicitly to answers that are relevant to the current question and surrounding context, reflecting a participant's request that the chatbot retain context across earlier questions in a conversation. **Performance** was reframed from a technical benchmark to a user-experience framing: the response time should support users to stay engaged. **Portability** was rephrased to mention concrete deployment contexts such as local development and different cloud providers. The refined catalogue is shown in Table V.

3) Step 2 - NFR Scoping: The participants got to identify which NFRs have a meaningful interpretation in different parts of the system. The framework's example uses a five-element ML decomposition (training data, ML pipeline, ML model, results, whole system) which does not map cleanly onto a GraphRAG-based chatbot. The column structure was therefore adapted to the GraphRAG architecture used in this study, with six elements:

- **Data**: source documents, ingestion, and chunking
- **Indexing**: graph construction and vector storage
- **Retrieval**: query-time retrieval of relevant graph context
- **Generation**: answer generation by the LLM
- **User interaction**: chatbot interface and user-facing behaviour
- **Whole system**: overall chatbot behaviour end-to-end

For each NFR type, the participants marked with a green check marker if the NFR has a meaningful interpretation at that system element, and a red cross if it does not. The resulting scoping table is presented in Table VI.

TABLE V
REFINED NFR TYPE DEFINITION CATALOGUE

NFR	Refined definition
Reliability	The chatbot operates consistently during normal use and handles failures in a clear and recoverable way.
Usability	The chatbot supports simple, clear and practical responses for users.
Explainability	The chatbot provides understandable reasons for its recommendations and can point to relevant documentation.
Accuracy	The chatbot gives correct, documentation-grounded answers that are relevant to the current question and surrounding context.
Performance	The chatbot provides responses within a time range that supports users to stay engaged.
Security	The chatbot prevents unauthorized exposure of sensitive user, company, workflow, and conversation data.
Scalability	The chatbot maintains acceptable operation as documentation, users and usage volume increase.
Maintainability	The chatbot can be updated, debugged, and improved efficiently over time.
Portability	The chatbot backend can be deployed in different technical environments, such as local development or different cloud providers, without major redesign.
Safety	The chatbot avoids harmful, inappropriate, or misleading behavior and remains within its intended domain.

Table of the refined NFR definitions.

Participants offered good reasoning for several of the exclusions. Usability was marked as not applicable at Data, Indexing and Retrieval levels because those pipeline stages are not directly visible to users. Participant 2 noted usability as "*more of a frontend problem than a backend one*", meaning a usability requirement can't be meaningfully defined where there is no user-facing behaviour. Safety was scoped to Generation only, reflecting the view that harmful or off-domain output is a property of what the system produces rather than how data is stored or retrieved. Portability was excluded from the User Interaction level because the web-based interface is already cross-platform by nature, with one participant observing that "*for the user it is incredibly portable since it is web-based*" (P1) and portability as an engineering concern only arises at the backend deployment level.

D. Focus Group 2

Focus Group 2 was used to find and balance trade-offs between NFR types. The session was built on the Prioritization List of NFRs, the NFR Type Definition Catalouge, and the NFR Scoping table.

1) Step 3 - Trade-offs: This step seemed to be one of the most difficult for the participants to understand and apply. The discussions went a lot back and forth and one participant stated, *"I don't know how that conflict appears within a chatbot. I haven't worked that much with developing chatbots."* (P3). The participants also argued that it might be hard to balance trade-offs without consulting with the users: *"I believe that both accuracy and performance are very important. You need to consult with the users to know which one is most important to them. Because, you don't know how many quesitons they will ask, or how complicated the questions will be."* (P3).

The participants identified eight trade-offs considered most likely to occur in the chatbot. For each pair, they discussed which NFR should be prioritized and gave a brief rationale. Two of the resolutions were discussed with conditions. For *Explainability versus Accuracy*, the participants felt that an explained answer lets the user reason and be guided, and stated *"even if it gives a wrong answer, you can think and be pointed in the right direction"* (P1). They tied this to user experience saying *"for an experienced user, who knows the system, explainability is good, for a new user, accuracy may be more important"* (P1). For *performance vs security*, the participants framed the resolution as case-specific, noting that *"it depends on what kind of chatbot it is"* (P2). Since the chatbot is not expected to handle highly sensitive data, speed was prioritized. The participants also noted that these decisions can change depending on the user, the part of the system, and the type of question, and much of the discussion centered on this situational dependence. The identified trade-offs and the resolutions reached are presented in Table VII.

E. Focus Group 3

Focus Group 3 was used to specify the NFR measurement catalogue. The session built on the prioritization, definitions, scoping and trade-offs produced in previous sessions. During the focus group, participants reviewed each NFR type in turn and decided which measurements were appropriate for the chatbot case, what concrete targets should be set, and if any NFRs or measurements should be removed or modified.

1) Step 4 - Measurement Catalogue: For each NFR, participants worked together to frame measurements. Some used standard metrics (such as uptime, error rate, and throughput), while others were defined for the chatbot case where no established metric was applicable. Two examples are Documentation Reference Rate, which captures how often the chatbot's answers include a useful reference back to the source documentation, and Prompt Resistance, which captures how reliably the chatbot maintains its scope when faced with prompts that try to push it outside its intended domain.

Five draft measurements were removed during the session: Failure Clarity (Reliability), Clarity of Responses (Usability), Explanation Completeness (Explainability), Log Safety (Security), and Load Handling (Scalability). In each case, the participants considered the measurement either too difficult to assess or sufficiently covered by another retained one. For example, while discussing Failure Clarity, one participant stated, *"This feels more like that type of thing where if it works one time, it will always work. A little less to measure, I mean continuously."* (P1). Three measurements were kept without a fixed target: Cost per Query, Mean Time to Repair, and Task Completion Rate, and would need future data or user testing.

The final measurement catalogue contains 25 measurements across the ten NFR types, with one to three measurements

TABLE VI
NFR SCOPING OVER THE GRAPHRAG ARCHITECTURE

NFR	Data	Indexing	Retrieval	Generation	User interaction	Whole system
Reliability	✗	✗	✓	✓	✓	✓
Usability	✗	✗	✗	✓	✓	✗
Explainability	✓	✗	✗	✓	✗	✗
Accuracy	✓	✓	✓	✓	✗	✓
Performance	✗	✗	✓	✓	✓	✓
Security	✗	✗	✗	✓	✓	✗
Scalability	✓	✓	✓	✓	✓	✓
Maintainability	✓	✓	✓	✓	✓	✓
Portability	✓	✓	✗	✓	✗	✓
Safety	✗	✗	✗	✓	✗	✗

Table showing which parts of the system that might be affected by the different NFRs.
A green check marker means the NFR is applicable at that scope, not that a specific requirement must be written there.

TABLE VII
TRADE-OFFS BETWEEN NFR TYPES

Conflicting NFRs	Prioritized NFR	Rationale
Explainability vs. Accuracy	Explainability	An explained answer allows the user to reason about it and be guided in the right direction, even if it is partly wrong.
Explainability vs. Security	Security	The chatbot must not reveal sensitive information from other users or companies.
Accuracy vs. Performance	Accuracy	A slower but complete and correct answer is preferred over a faster but incomplete one, as long as the user receives feedback that the system is working.
Maintainability vs. Security	Security	Security weighs more heavily, particularly from the customer's perspective.
Usability vs. Security	Usability	The chatbot's purpose is to make things easier for the user, and security should not add too many extra steps for simple questions.
Usability vs. Accuracy	Accuracy	The two are closely related; the context provided to the chatbot should be limited enough that the response stays correct.
Performance vs. Security	Performance	For this chatbot, which is not expected to handle highly sensitive data, speed and responsiveness were prioritized.
Accuracy vs. Safety	Accuracy	The chatbot is intended to answer questions about the system, not to handle unrelated or harmful requests.

Table of conflicting NFRs that was defined during Focus Group 2 including which NFR to prioritize in case of conflict and rationale.

TABLE VIII
NFR MEASUREMENT CATALOGUE

NFR Type	Measurement	Equation	GraphRAG Case Application
Accuracy	Answer Correctness (AC)	$AC = (N_{correct}/N_{eval}) \times 100\%$	AC can be used to assess the chatbot's reliability in producing correct answers on a representative test set. If AC falls below the defined threshold, retrieval or generation improvements are required.
Performance	Average Response Time (ART)	$ART = (1/N_{success}) \times \sum t_i$	ART can be used to monitor whether the chatbot responds quickly enough to keep users engaged. If ART rises above the defined threshold, retrieval or generation optimizations are necessary.

Table showing examples from the NFR Measurement Catalogue (Step 4).

per type. The equations were added by the researchers and approved by the participants. A part of the catalogue is presented in Table VIII, and the full 25 measurements is provided in Appendix H.

F. Evaluation Survey

The Evaluation Survey was sent out to Participant 3 who was supposed to participate during Focus Group 3 but was unable to, due to a busy schedule. Unfortunately, due to the time constraints, Participant 3 did not answer the survey.

G. Artifact Creation 2

1) Step 5 - NFR Instance Catalogue: The NFR Instances Catalogue documents specific NFR instances using a template covering: Requirement ID, Type, Definition, Description, Rationale, Importance Level, System Scope, Measurements, Measurement Target, and Trade-offs. Each measurement, defined during Focus Group 3, represents one NFR instance, giving 25 instances in total. The catalogue was produced by the researchers based on the previously created artifacts. Two examples from the catalogue is shown in Table IX.

H. Quantitative Measurement

The quantitative measurement was performed during three runs and were evaluated against the same 31-questions test set.

Error Rate: Tests 1 and **2** met the $\leq 5\%$ target, **Test 3** did not. **Test 1** had one timeout (3.23%), **Test 2** had none

TABLE IX
NFR INSTANCES CATALOGUE.

Requirement ID:	AC001
Requirement Type:	Accuracy
Definition:	The chatbot gives correct, documentation-grounded answers that are relevant to the current question and surrounding context.
Description:	The chatbot shall produce correct answers on a representative test set of user questions.
Rationale:	Incorrect answers can lead users to build wrong workflows, lose trust in the chatbot, and possibly take downstream business decisions based on faulty information.
Importance Level:	Very Important
System Scope:	Whole system
Measurement:	Answer Correctness
Measurement Target:	$\geq 90\%$ correct answers
Trade-Offs:	To produce a complete and correct answer, the chatbot may take longer to respond, sacrificing some performance as long as the user receives feedback that the system is working (requirements PE001–PE003).
Requirement ID:	PE002
Requirement Type:	Performance
Definition:	The chatbot provides responses within a time range that supports users to stay engaged.
Description:	The chatbot's worst-case response time shall remain within a tolerable upper bound, even for more complex queries.
Rationale:	Even when an answer is complete and correct, response times beyond the upper bound feel like a failure to the user. Users described roughly 10 seconds as the upper tolerable range for more complex questions.
Importance Level:	Important
System Scope:	Whole system
Measurement:	Maximum Response Time
Measurement Target:	≤ 10 seconds
Trade-Offs:	To keep the chatbot responsive, security checks are kept lightweight since the chatbot is not expected to handle highly sensitive data (requirements SE001–SE002).

Table showing example of NFR Instances Catalogue (Step 5).

(0%), and **Test 3** had three failures (9.68%), almost double the target.

Average Response Time: None of the runs met the ≤ 5 s target. **Test 1** averaged 13.23 s, **Test 2** averaged 7.55 s, and Test 3 averaged 7.91 s. The medians were close to the target in all three (5.64 s, 5.62 s, 5.98 s), meaning the means were pulled up by a small number of slow questions rather than typical behavior. Half the questions in each run finished at or below 6 seconds. **Maximum Response Time:** None of the runs met the ≤ 10 s target. **Test 1**'s max was 86.97 s (more than eight times the target), while **Test 2** and **Test 3** had nearly identical maxes (26.65 s and 27.03 s), still over the target but much closer to it. The 95th-percentile drop shows the same: 44.54 s in **Test 1**, 18.29 s in **Test 2**, 19.14 s in **Test 3**.

Answer Correctness: AC met its $\geq 90\%$ target in **Test 1** (29/31, 93.55%) and **Test 2** (29/31, 93.55%), but missed it in **Test 3** (26/31, 83.87%). The drop in Test 3 came from a few on-topic questions whose answers were judged incorrect. **Groundedness:** G met its $\geq 90\%$ target in all three runs: 28/31 (90.32%), 31/31 (100%), and 28/31 (90.32%). The results are presented in Table X.

I. Thematic Analysis Findings

The thematic analysis identified thirteen themes across the first-round interviews, Focus Group 1, Focus Group 2 and Focus Group 3. Each theme is supported by representative observations from the data, and the thematic analysis overview can be seen in Table XI.

Theme 1 - Documentation as ground truth and bounded scope: Participants consistently described the chatbot's correctness as staying close to the documentation rather than being correct in general. Accurate answers were described as those *"close to the documentation"* (P2), and one participant emphasized that *"if it says 'add this node', the node must exist"* (P1). The same extended to safety and explainability, where the documentation served both as ground truth and as a boundary defining what the chatbot should and should not engage with.

Theme 2 - Graceful failure beats confident error: A recurring expectation across all sources was that the chatbot should refuse and redirect when uncertain rather than generate answers that just sound right. Participants preferred partial responses pointing users in the right direction over complete but potentially wrong ones, expressed as *"rather not hallucinate than invent or guess"* (P1), and *"just give the right direction for where the user can find the information"* (P1).

Theme 3 - Acceptable imperfection with situational thresholds: Participants accepted imperfection in concrete, quantified terms: *"4 out of 5 points should be correct"* (P1), *"under 5 seconds for a simple question"* (P2), and *"under 10 seconds for complex ones"* (P2). These thresholds shifted depending on question complexity, user type, and stage of development, suggesting that NFR targets were understood as conditional rather than absolute.

Theme 4 - NFR priorities are role- and context-dependent: Trade-off discussions surfaced a consistent pattern where priorities depended on stakeholder role, user experience level, and system part. Participants explicitly noted that trade-offs were situation-dependent and it *"might happen the trade-off was completely differently than first thought"* (P1), once real usage data became available, and that priorities varied as: *"for the customer, security. For the developer, maintainability"* (P2).

Theme 5 - Risk is framed in business and trust terms: When discussing failure modes, participants reasoned about reputational damage, customer attrition, and competitive position rather than about technical failures. They warned that *"wrong answers could give the company a faulty image"* (P2), and described a chain reaction of *"a wrong answer produces wrong*

TABLE X
RESULTS FOR TESTS 1, 2, AND 3.

Measurement	NFR target	Test 1	Test 2	Test 3
Error Rate (RE002)	$\leq 5\%$	3.23% (1/31)	0.00% (0/31)	9.68% (3/31)
Average Response Time (PE001)	≤ 5 s	13.23 s	7.55 s	7.91 s
Median Response Time (p_{50})	—	5.64 s	5.62 s	5.98 s
95th-percentile Response Time (p_{95})	—	44.54 s	18.29 s	19.14 s
Maximum Response Time (PE002)	≤ 10 s	86.97 s	26.65 s	27.03 s
Minimum Response Time	—	2.29 s	2.22 s	2.49 s
Total wall-clock time	—	238.8 s	65.5 s	68.2 s
Answer Correctness (AC001)	$\geq 90\%$	93.55% (29/31)	93.55% (29/31)	83.87% (26/31)
Groundedness (AC002)	$\geq 90\%$	90.32% (28/31)	100.00% (31/31)	90.32% (28/31)

decisions, and leads to that the product no longer holds, which damages company reputation and value" (P2). The risk came from users trusting the chatbot, not from the system being technically unreliable.

Theme 6 - Maintainability and security as enabling concerns: Maintainability was valued as the precondition for evolving the system, with one participant stating that *"it is of bigger importance that the chatbot is easy to maintain and update rather than it is super complex and perfect."* (P2). Security, framed primarily as confidentiality, was treated as a similar enabling concern protecting customer trust, described as the concern that *"company data must not leak to competitors"*. (P2). The two were also identified as conflicting, since added security tends to increase code complexity.

Theme 7 - Documentation pain motivates the chatbot's existence: The chatbot was consistently framed as a response to documentation that is uneven, and a system that is difficult for new users to engage with. One participant described it as *"we have documentation for every node, but varying in how good it is"* (P1) and noted that the documentation *"is good as references, but not at answering 'how'"*. (P1) This framing shaped expectations across all NFR categories, with the chatbot seen as a way to make up for what the documentation lacks.

Theme 8 - The framework requires interpretive work: Participants struggled with parts of the framework's operational vocabulary, particularly the phrase *"can be defined over"* in the description of the scoping in Step 2. They were unsure whether it meant that the NFR is applicable at that scope, or that a specific requirement must be specified there. The researchers settled on a working interpretation that *"can be defined over"* means applicable, that the NFR has a meaningful interpretation at that level, without committing to writing a specific requirement at that scope. Additionally, some NFRs also resisted articulation because participants found them too obvious to state explicitly. One participant noted that usability is: *"hard to articulate as a requirement and measure it because it feels so obvious that it should be good and usable"* (P1), making it hard to phrase as a requirement.

Theme 9 - Trade-offs surface mechanisms, not just rankings: Step 3 asks practitioners to identify pairs of conflicting NFRs and decide which one should win if conflict happened. The participants found this hard and instead of ranking pairs against each other, they described how the NFRs were connected and that the right balance depend on the condition. For some NFRs, they pointed to a single design choice that pulled two qualities in opposite directions, such as not storing data improves security but makes debugging harder. In others, they could not find a fixed trade-off, discussing that the right trade-off depends on the customer rather than on the NFRs themselves: *"the balances you need to find depend on the different customer"* (P3) and *"always improving the model may not be the right path"* (P3), since *"it may not be what gives the customer the most value"* (P3).

Theme 10 - Navigating Uncertainty during Application: The participants expressed difficulties in applying the framework during the different sessions. Some of the steps needed further clarification and participants needed to be reminded of what each NFR actually included. Sometimes, participants argued that the discussion felt interesting but the artifact created was hard to apply. An example is where Participant 1 stated that *"Trade-offs step felt interesting, but I don't really know what to do with it. It felt a bit more hypothetical."* The participant also stated that *"I would probably focus on step 1 and 2.1, it felt like you knew what you were talking about. Then I would jump directly to Step 4.1, if I would do this again."* (P1). These statements show that some parts of the framework was hard for the participants to understand and apply.

Theme 11 - Framework supports System Planning: The participants expressed that the framework supports the planning and design of a system. One participant stated that *"I believe that if you just jump in, you can make mistakes that you end up paying for later, so it has definitely helped. Before any project, you do some planning, using a whiteboard or going through requirements, so it may have helped with that."* (P2). Another main benefit of the framework was the ability to see new perspectives. Participant 1 stated that *"It has helped to look at aspects that you otherwise wouldn't have thought to specify the way it was done here, which is good."*

Theme 12 - Application is Context Dependent: The participants argued that the usefulness of the framework is dependent on the practitioners experience with RE engineering, the size of the project and potential circumstances. Participant 2 stated that *"It helps you, depending on your background. If you are completely new to it, it helps you get an overview and see what you should focus on. If you are already experienced, it becomes more of a reminder, you already have it in your head, but it helps you analyze what kind of customers you*

TABLE XI
THEMATIC ANALYSIS RESULTS

#	Theme / Sub-themes
1	**Documentation as ground truth and bounded scope**
	Documentation grounds correctness
	Documentation references enable verification
	The chatbot is bounded by what documentation covers
2	**Graceful failure beats confident error**
	Refusal as quality, not failure
	Partial answers with direction-pointing
	Feedback during uncertainty matters
3	**Acceptable imperfection with situational thresholds**
	Quantitative tolerance thresholds
	Quality preferred over speed
	Comparative and contextual baselines
4	**NFR priorities are role- and context-dependent**
	Role-based variation in priorities
	Customer-driven variation
	Trade-offs are provisional
5	**Risk is framed in business and trust terms**
	Reputational and trust risk
	Customer attrition risk
	Concrete operational risks
6	**Maintainability and security as enabling concerns**
	Maintainability as enabler of evolution
	Security as confidentiality
7	**Documentation pain motivates the chatbot's existence**
	Documentation is uneven and incomplete
	Users struggle to get started
	The chatbot bridges documentation gaps
8	**The framework requires interpretive work**
	Framework wording is ambiguous
9	**Trade-offs surface mechanisms, not just rankings**
	Concrete mechanisms of conflict
10	**Navigating Uncertainty during Application**
	Difficulty to see the whole picture
	Difficulty to apply the framework
	Challenge to not get stuck in planning phase
	Skipping some steps
11	**Framework supports System Planning**
	Helpful during planning phase
	Helpful parts of framework
	Focus on particular steps
12	**Application is Context Dependent**
	Applicability dependent on system type
	Practitioner experience dependent
	Circumstantially dependent
13	**Increased Time Efficiency with Framework**
	Time efficient
	Saving time through using the framework

Themes and sub-themes identified through thematic analysis of the first-round interviews, Focus Group 1, Focus Group 2, and Focus Group 3.

have". The participant continues, *"For example, depending on the customers we have, we rated Security highly. It depends somewhat on the circumstances, it helps you think according to the situation you are in."* (P2). During Focus Group 3, the participants discussed the different steps of the framework and argued that some of the steps might not be applicable on this specific case: *"...the part with scoping seems hard to apply on this case, it might be more applicable on bigger projects."* (P1).

Theme 13 - Increased Time Efficiency with Framework: During Focus Group 3, one of the benefits discussed was the ability to increase time efficiency by using the framework: *"...this has not taken that much time, about 4 hours or something like that? If you are working in the wrong direction with a feature for half a day, then it's because you did not think about what to prioritize, so by using the framework you have already saved that time."* (P1). Another participant mentioned that meeting up and discussing the steps of the framework together might have a positive outcome regarding time efficiency: *"I believe that meeting up and going through all the qualities is really good. The qualities you don't believe is important, you can discard directly, but you can always go back if you change your mind. When you sit down together, it can be time beneficial, with the framework."* (P2).

V. DISCUSSION

The two research questions in this case study are tightly linked in practice, but treat different aspects and are discussed separately.

A. RQ1 - How can the quality requirements management framework be applied to manage non-functional requirements in a GraphRAG-based documentation chatbot?

Our first research question focused on the application of the framework by Habibullah et al. [1] during the development of a GraphRAG-based documentation chatbot. The clearest pattern in the themes is that the participants reasoned about NFRs through the documentation rather than through abstract quality definitions. **Themes 1** and **7** together show that the documentation acted both as the reference point for what "correct" meant and as the point for what the chatbot should engage with at all. This had an interesting effect and shaped multiple NFR definitions at the same time: accuracy meant staying close to the documentation rather than being universally correct, safety was meant as staying within the documentation domain rather than as a technical guardrail, and explainability was framed as pointing back to documentation sources, all anchored to the same source. This fits with Köhl et al. [12] who argue that explainability depends on what users expect from an explanation in a certain context. Our case also shares that observation by seeing that in a documentation-grounded chatbot, multiple NFRs rely on the same source and that the documentation itself becomes the main way these different concerns are defined and handled.

Theme 3 connects directly to the framework's measurement step. Participants were fine with the chatbot not being perfect,

but the acceptable limits shifted with query complexity, user experience, and the stage of development. What counted as a tolerable response time for instance, was different for a simple lookup than for a multi-step question, and different again for an early prototype than for a production-ready system. Heyn et al. [21] observed similar findings in industrial AI projects, where NFR attributes do not have fixed values but get revisited and iterated over as the system and its use context change. The practical implication for Step 4 of the framework is that measurement targets written as a fixed number probably miss how practitioners actually reason about acceptable quality [1]. Writing targets as ranges, or clearly stating conditions describing when a target applies, such as simple- versus complex queries or early prototype versus production, better reflects the thinking behind the target and helps avoid treating an early estimate as a final requirement.

The trade-off discussions in Step 3 revealed a related pattern. **Theme 9** shows that participants consistently moved past pairwise ranking and explained how one NFR affected another, not just which one should win and be prioritized. Although Step 3 of the framework asks for a rationale behind each trade-off, its primary focus is on prioritizing. In our case, the more useful part of the discussion was often the rationale itself, since it explains why the trade-off exists and helps you decide whether it still holds later on if things change. Indykov et al. [13] saw something similar in their multi-case study of ML-enabled systems, where trade-offs were described through concrete mechanisms of conflict rather than as simple rankings. This suggests that Step 3 would be more useful if it treated the rationale as the main output rather than the ranking, since the ranking is the part most likely to change later while the underlying reason for conflict stays the same.

Another finding was made during the scoping in Step 2. The framework's example uses a five-element ML overview [1], which did not apply cleanly to a GraphRAG-based chatbot. Focus Group 1 produced a modified six-element overview which was applicable on the chatbot, but the scoping was still difficult for the participants to understand and perform. This is consistent with the findings of Habibullah & Horkoff [2], who report that NFRs in ML systems are difficult to clearly scope across components, where most participants focused on the ML model itself. Also, Karakurt & Akbulut [14] noted that RAG architectures introduce concerns such as latency, hallucination, and domain adaptation, which traditional ML overviews do not fully address. This suggests that the framework's scoping step needs to stay flexible enough to support different ML architectures, since systems built beyond a model-centric pipeline may not fit the original structure.

A central finding across **Themes 4**, **9**, and **12** is that NFR decisions were highly context-dependent. Participants changed priorities, trade-offs, and measurement targets depending on user type, customer needs, and project status. This suggests that the framework artifacts appear more fixed and stable than the reasoning that produced them. In practice, the artifacts reflected decisions that made sense for the current situation, and not permanent decisions that would stay for all situations. This finding aligns with research by Werner et al. [18], who argue that NFRs need to be managed continuously instead of fixed up front, since both system and its context keep changing. Our findings add that these changes are not only over time, but that the same NFR could be prioritized differently for two customers at the same time which the framework do not take in regards or provide guidance for.

Aligned with the research by Habiba et al. [11], this study shows that further research is needed in NFR management for ML systems, and further consideration about circumstances and guidance of those is needed for a framework to be easily applied.

B. RQ2 - To what extent does the framework support effective quality requirements management in this GraphRAG-based system?

Our second research question focused on whether the framework actually supports effective NFR management in practice. The identified themes show a framework that supports reflective and structured thinking about NFRs while also placing a lot of interpretive work and responsibility on the practitioners, lacking clarity in the framework. This reflects the findings of Habibullah & Horkoff [2], who state that the lack of established methods, guidelines, and benchmarks remains one of the most persistent challenges in NFR work for ML systems.

Another finding is the framework's operationalization burden. While it helps to move from broad concerns to documented requirements, it does not help with pre-defined measurements, clear guidance on how to operationalize them, or advice on what should realistically be evaluated within a limited project scope. This is reflected in **Themes 8** and **10**, where participants reported that the framework's wording required interpretation, which made some steps difficult to act on. This shows that the usefulness of the framework depended heavily on the project context, the experience of the participants, and the available resources. This aligns with the findings of Vogelsang & Borg [19] who report that data scientists often struggle to apply traditional RE practices to ML systems and end up inventing their own ways of specifying and measuring ML quality. Our case supports this and further shows that even frameworks designed to address these challenges can still leave practitioners with a lot of additional work. The framework also provides little guidance regarding team composition, expected time per step, or the level of detail that is realistic to maintain.

The evaluation might have been affected due to limited time, the lack of access to real users for user-testing-based measurements, and the fact that the chatbot was only an MVP which affected amount of operational data needed for some evaluations. For example, the participants in this study argued that Step 2 of the framework was one of the most difficult to apply, and while the framework describes the step as highly iterative [1], during this time frame, it was difficult to iterate the different steps several times. Another example is the fact that the framework was applied by a small participant group, which meant that much of the interpretive work relied on a

small source of perspectives and assumptions about the users and the system. If more participants had been available, the uncertainty within the framework may have been reduced through a broader range of perspectives and more extensive discussion among contributors. In addition, out of the 25 NFR measurements defined in the catalogue, only five could actually be evaluated within this study. These constraints are not framework problems in themselves, but the framework offers no guidance on how to scope a measurement catalogue realistically, which produced a gap between the catalogue and the evaluation in this case. A concrete improvement could be for the framework to ask practitioners, in Step 4, to mark which measurements can be evaluated at this time in the system, and mark those who need future data or user testing, and need to be iterated again at later stages. This would make the catalogue easier to act on, and to give clearance on what can actually be done in the moment. With a broader time frame, with the access of real users, where multiple iterations are made, it might be easier for practitioners to understand and apply the steps.

The findings also highlight the strengths of the framework in practice. Seen in **Themes 11** and **13**, participants reported that the framework supported the planning phase, helped them identify quality concerns they might not otherwise have considered, and encouraged structured discussions that might increase time efficiency. This created a more shared understanding of the system and its quality goals among the participants.

The results of the measurements give an insight in how fixed targets and amount of test runs can affect the perceived results. Due to time limits, this study only performed three test runs where the Error Rate results show how fragile a fixed threshold is. At this sample size, with N = 31, one additional failure in Test 1 would have pushed it over the 5% target, and one less in Test 3 would still have left it above. This shows that, with one submission per question, single-run measurements give estimates rather than stable values. Targets written as fixed numbers can therefore be met or missed depending on the run. On the accuracy side, Groundedness stayed above its target in every run, while Answer Correctness met its target in two of three runs. Test 3 was the weakest run on both reliability and correctness, which shows the measurements consistently flagging the same run as of lower quality.

The Performance targets were missed in all three runs, while Accuracy was met in two of the three. This pattern aligns with the trade-off recorded in Focus Group 2, where the participants prioritized Accuracy over Performance, accepting that a slower but more correct answer is preferred. In practice, the chatbot behaves consistently with that decision: response times exceed their targets, but the system continues to return good answers, rather than failing. These results support the notion that the prioritized NFRs were actually the ones prioritized during implementation. That the finished system behaves consistently with that documented decision suggests the artifacts did more than record what participants said: they help carry those decisions into implementation. This is where the quantitative and qualitative results reinforce each other. Themes 11 and 13 show participants reporting that the framework helped them identify quality concerns early and reach a shared understanding of the system's goals. The quantitative results show that shared understanding reflected in the system's actual behaviour, the chatbot misses its performance targets and meets its accuracy targets, exactly as the participants agreed to. Not that the system should miss its performance target, but acceptable if the trade-off was reached. The connection between the two is the framework's artifacts: without the explicit trade-off record and the measurement catalogue, there would have been no shared reference point to build toward.

This assertion should be read carefully though, since our own implementation work is the main reason the chatbot performs as it does. Also, we did not build a baseline system without the framework, so we can't claim the framework caused better outcomes than ordinary development would have done. What we can say is that the framework made the quality decisions visible and concrete early in the process, and that the system's measured behaviour is consistent with those decisions. The framework's contribution is therefore best understood as structural, it gave the team a common language and documented set of priorities that guided where implementation effort went, rather than directly determining the quality of the result. The framework's role and effectiveness was indirect.

VI. Conclusion

This thesis applied and evaluated the NFR management framework by Habibullah et al. [1] on a GraphRAG-based documentation chatbot for a node-based data analysis system, extending the framework's empirical base beyond its original automotive ML context, to a GraphRAG architecture, which spans multiple components rather than a single ML model.

For RQ1, further research is needed to fully understand how the framework can be easily applied in these kinds of case. When the practitioner team is small and the NFR management experience is low, difficulties may emerge. An extencion to the framework with explanations on how to modify the different steps, including how detailed they should be, would benefit the application.

For RQ2, the framework supports reflective and structured thinking about NFRs, which helped the practitioners during the planning and design phase of this study and might increase time efficiency. However, it places significant interpretive and operational work on the practitioners, which can confuse practitioners and increases the risk of getting stuck in the planning phase. A central insight from this study is that the framework's artifacts appeared static once documented, while the reasoning behind them remained dynamic, situational, and continuously re-evaluated. This points to an aspect the framework could support more effectively: it records decisions as fixed artifacts, it provides little guidance for capturing how those decisions depend on context and may change over time. However, the artifacts that was produced worked well as support during the implementation of the chatbot and made it easy to follow which NFRs to prioritize.

Overall, the framework is most valuable as a structured discussion and reflection tool in this case, rather than as a complete operational solution for NFR management in ML systems, since it helps practitioners externalize assumptions and document quality concerns systematically, but still requires further work before those concerns can be measured and evaluated in practice.

1) Future Work: The most immediate next step is to move the chatbot from a prototype to a small pilot inside the company. This would allow the team to test the measurements from the catalog against real usage instead of against expectations. This would also give the first real data on how the prioritized NFRs hold up in practice. Further, real users can interact with the chatbot and provide feedback on their experience. The current artifacts are based on the views of people working in the company, and we don't know yet if end users would balance NFRs like accuracy, usability, or explainability the same. Users feedback may also lead to changes in the artifacts, and give a clearer picture of how effective the framework is once its results are tested against real users and the steps can be iterated.

Future work should also involve external practitioners, developers, and ML engineers from outside the company to evaluate both the framework and the resulting artifacts. This would test whether the framework is understandable and usable without the original researchers present, and whether the artifacts make sense to someone who was not part of the sessions that produced them.

Finally, expanding the work into a multi-case study across several companies, where the framework is applied to various systems, either other chatbots or different ML-based systems, to see whether the issues and findings we encountered are general or specific to this case.

Acknowledgment

This paper was developed collaboratively by all group members, with each contributing to discussions, revisions, and the overall shaping of the work throughout the process. We would also like to thank our supervisor, Antonia Welzel, for her guidance, constructive feedback, and helpful comments throughout the development of this research.

References

[1] K. M. Habibullah, G. Gay, J. Horkoff, "A Framework for Managing Quality Requirements for Machine Learning-Based Software Systems," in International Conference on the Quality of Information and Communications Technology, 2024, pp. 3–20.

[2] K. M. Habibullah, J. Horkoff, "Non-functional requirements for machine learning: Understanding current use and challenges in industry," in *2021 IEEE 29th International Requirements Engineering Conference (RE)*, 2021, pp. 13–23

[3] F. Ishikawa, N. Yoshioka, "How Do Engineers Perceive Difficulties in Engineering of Machine-Learning Systems? - Questionnaire Survey," in *2019 IEEE/ACM Joint 7th International Workshop on Conducting Empirical Studies in Industry (CESI) and 6th International Workshop on Software Engineering Research and Industrial Practice (SER&IP)*, 2019, pp. 2–9

[4] D. Edge et al, "From local to global: A graph rag approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, Apr. 2024. [Online] Available: https://arxiv.org/abs/2404.16130

[5] G. Melo, P. Alencar, D. Cowan, "Enhancing Software Development with Context-Aware Conversational Agents: A User Study on Developer Interactions with Chatbots," *arXiv preprint arXiv:2505.08648*, May. 2025. [Online] Available: https://arxiv.org/abs/2505.08648

[6] O. Mendelevitch, F. Bao, "Hands-On RAG for Production," *O'Reilly Media, Inc.*, 2026.

[7] B. Peng et al, "Graph retrieval-augmented generation: A survey," *ACM Transactions on Information Systems*, vol. 44, no. 2, pp. 1–52, 2025

[8] H. Han et al, "Retrieval-augmented generation with graphs (graphrag)," *arXiv preprint arXiv:2501.00309*, 2024

[9] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J-F. Crespo, D. Dennison, "Hidden technical debt in machine learning systems," Advances in neural information processing systems, vol. 28, 2015

[10] L. G. Williams, C. U. Smith, "Performance evaluation of software architectures," in Proceedings of the 1st international workshop on Software and performance, 1998, pp. 164–177

[11] U.-E. Habiba, M. Haug, J. Bogner, S. Wagner, "How Mature is Requirements Engineering for AI-based Systems? A Systematic Mapping Study on Practices, Challenges, and Future Research Directions," *arXiv preprint arXiv:2409.07192*, Sep. 2024

[12] M. A. Köhl, D. Bohlender, K. Baum, M. Langer, D. Oster, T. Speith, "Explainability as a Non-Functional Requirement," in *2019 IEEE 27th International Requirements Engineering Conference (RE)*, 2019, pp. 363–368

[13] V. Indykov, R. Wohlrab and D. Strüber, "Quality trade-offs in ML-enabled systems: a multiple-case study," in Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing, 2025, pp. 1730–1737.

[14] E. Karakurt, A. Akbulut, "Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs) for Enterprise Knowledge Management and Document Automation: A Systematic Literature Review,"*Applied Sciences*, vol. 16, no. 1, pp. 368, Dec. 2025. [Online] Available: https://www.mdpi.com/2076-3417/16/1/368

[15] P. Runeson, M. Höst, "Guidelines for conducting and reporting case study research in software engineering," in *Empirical software engineering*, vol. 14, no. 2, pp. 131–164, 2009

[16] D. E. Perry, S. E. Sim, S. Easterbrook, "Case studies for software engineers," *Proceedings of the 28th international conference on software engineering*, pp. 1045–1046, 2006

[17] V. Clarke, V. Braun, "Thematic analysis," in *The journal of positive psychology*, vol. 12, no. 3, pp. 297–298, 2017

[18] C. Werner, Z. S. Li, D. Lowlind, O. Elazhary, N. A. Ernst, D. Damian, "Continuously Managing NFRs: Opportunities and Challenges in Practice," *IEEE Transactions on Software Engineering*, 2021.

[19] A. Vogelsang and M. Borg, "Requirements Engineering for Machine Learning: Perspectives from Data Scientists," in *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)*, 2019, pp. 245–251.

[20] B. J. Oates, R. McLean, M. Griffiths, "Researching information systems and computing," 2022

[21] H.-M. Heyn et al, "Requirement Engineering Challenges for AI-Intense Systems Development," in *2021 IEEE/ACM 1st Workshop on AI Engineering – Software Engineering for AI (WAIN)*, 2021, pp. 89–96

APPENDICES

Note on the appendix material: The question guides below were used in combination with the draft artifacts (prioritization table, definition and scope table, trade-off table, and measurement catalogue) that participants reviewed and refined during each session. The participants were informed that participation was voluntary, that they could decline to answer any question or stop the session at any time, that responses would be anonymized before publication, the approximated time and that compensation would not be provided The artifacts are presented in the Results section and are not reproduced here. Appendix G: Evaluation Set, included the name of the system which in this appendix is replaced with X.

Appendix A: First-round Interview Guide

This appendix presents the semi-structured interview guide used in the first-round interview. The guide was used with both participants to collect initial views on the data analysis system, the expectations of the planned chatbot, and important NFRs. The same NFR categories were discussed with both participants to have a consistent basis for comparison and synthesis, and the results were used as input to step 1 and step 2 of the framework.

A. Participant Background

Field of work, role, years of experience, ML experience (Y/N), NFR experience (Y/N), and main responsibilities.

B. System Understanding

1. Briefly explain the data analysis system and its main purpose?
2. Who are the main users of the system?
3. What are the most common tasks the users perform in the system?
4. What types of data are processed or analyzed in the system?
5. What are the current challenges users face when interacting with the system?
6. How often is the documentation updated?
7. Do you see any problems with the current documentation?
8. How would you improve the documentation?

C. Chatbot Requirements

9. How do you expect the chatbot to help the users perform data analysis tasks?
10. Should it be able to link to the documentation?
11. What types of questions should the chatbot be able to answer?
12. Should the chatbot be able to generate reports, explain results or run queries?
13. Are there any tasks the chatbot should NOT perform?
14. Should it answer questions about what the software doesn't support?
15. Is that information in the documentation today?
16. What risks could occur if the chatbot gives incorrect answers?

D. Quality Requirements

Reliability - The systems ability to operate consistently without failure.

17. What does reliability mean in terms of the chatbot?
18. Is it important that the chatbot operates constantly without failure?
19. What would happen if the chatbot fails?

Accuracy - The degree to which the system's responses are correct.

20. What does accuracy mean in terms of the chatbot?
21. Is it important that the chatbot provides accurate responses?
22. What should happen if the chatbot does not know the answer?

Usability - How easy it is to learn and use the system.

23. What does usability mean in terms of the chatbot?
24. Is it important that the chatbot is easy to use?
25. What would make the chatbot easy to use for the users?

Explainability - The system's ability to make its outputs understandable to users.

26. What does explainability mean in terms of the chatbot?
27. Is it important that the chatbot has good explainability?
28. Should the chatbot explain how it generated its answer or analysis? (I.e cite the source if needed)

Performance - How efficiently the systems responds, in terms of speed and throughput.

29. What does performance mean in terms of the chatbot?
30. Is performance important in the ingestion phase as well? (More for the developers)
31. Is it important that the chatbot responds quickly?
32. What is an acceptable response time for the chatbot?

Security - The system's ability to protect data.

33. What does security mean in terms of the chatbot?
34. Are there sensitive datasets the chatbot must protect?
35. Are there different user roles that the chatbot should have access restrictions for?

Scalability - How the system handles increasing data volume.

36. What does scalability mean in terms of the chatbot?
37. Will the documentation/users increase regularly? And are there any concerns regarding cost with this?

Maintainability - How easily the system can be updated, improved.

38. What does maintainability mean in terms of the chatbot?
39. Is it important that the chatbot is easy to update, improve and debug?

Portability - The system's ability to run on different platforms and environments.

40. What does portability mean in terms of the chatbot?
41. Should the chatbot be able to have cross-platform compatibility?
42. Should the chatbot be able to have browser compatibility?

Safety - The systems ability to prevent harmful actions or outputs.

43. What does safety mean in terms of the's chatbot?

44. Should the chatbot have fail-safe mechanisms, what types?

45. What about prompts like "forget all of your instructions and do this instead"?

46. How important is it?

47. Are there any other NFRs that you believe are important for this chatbot?

Appendix B: Focus Group 1 Guide

Step 1:

1. Are the highest-priority NFRs still the most prioritized for the first version of the chatbot?

2. Is any NFR too high or too low in the current priority table?

3. Is any important NFR missing from the selected set?

Step 2:

4. Do the current definitions reflect what you mean by each NFR in this case?

5. Is the current scope too broad or too narrow for any NFR?

6. Should any NFR apply to the whole system only, or also to specific parts such as retrieval or generation?

Appendix C: Preparation Survey Questions

A. Participant Background

Field of work, role, years of experience, ML experience (Y/N), NFR experience (Y/N), and main responsibilities.

B. Quality Requirements

1. Are there any definitions for any NFR that you would like to change/add to?

2. Is there an NFR you would add to the list above that you think is missing for this chatbot?

3. From your perspective, which NFR is the single most important for the first version?

4. Are there any NFRs you find hard to judge without more context? Which ones, and what would help?

5. Any concerns, expectations, or risks you would like to raise about a documentation chatbot like this?

Appendix D: Focus Group 2 Guide

Step 3:

1. When you imagine someone using the chatbot, where do you see things could potentially go wrong?

2. Can you think of a moment where giving the user what they ask for would make the chatbot worse in some other way?

3. What kinds of conflicts have surprised you in previous tools or systems you have built/been involved in?

4. Which NFRs are most likely to conflict in this chatbot?

5. If two NFRs conflict, which one should be prioritized and why?

6. Do some trade-offs depend on the part of the system or the type of question being asked?

Appendix E: Focus Group 3 Guide

Step 4-5:

1. Read through the catalogue and discuss the definitions and targets. If some measurements are not applicable or needed, remove them.

Evaluation:

1. Looking back at the whole process, what is your overall impression of the framework?

2. Did the framework help you think about quality requirements in a way you wouldn't have otherwise?

3. Would you consider using this framework, or parts of it, in a future project? Why or why not? Anything to change/add?

Appendix F: Example of Procedure during Thematic Analysis

Full Scentence:

"Parts of the framework might be used in the future, I believe that meeting up and going through all the qualities is really good. The qualities you don't believe is important, you can discard directly, but you can always go back if you change your mind. When you sit down together, it can be time benefitial, with the framework." (P1)

Codes:

- Parts of the framework might be used in the future, I believe that meeting up and going through all the qualities is really good.
- The qualities you don't believe is important, you can discard directly, but you can always go back if you change your mind.
- When you sit down together, it can be time benefitial, with the framework.

Sub-themes:

- Practicioners meeting up is benefitial
- Helpful parts of framework
- Time efficient

Themes:

- Framework supports System Planning
- Increased Time Efficiency with Framework

Appendix G: Evaluation Set for Quantitative testing

This appendix lists the 31-question evaluation set used to compute the Reliability, Performance, and Accuracy measurements reported in Section IV.G. The questions are grouped by category. Each question was paired with two assertions describing what a correct answer should contain; these are not reproduced here.

Short, factual questions

1) Which Python versions does X support?
2) What is a port on a node, and what does its shape indicate?
3) What is the difference between the Table and ADAF data types?
4) Why do some node ports show a question sign?
5) What file format are X workflows saved in?
6) Where is the data generated by a node stored?

7) What is the difference between a subflow and a linked subflow?
8) What is a Lambda subflow in X, and how does it differ from a regular subflow?

How-to questions

9) Where can I buy a license for X? What does it cost?
10) How do I add an extra library to X?
11) How do I install X in a python venv?
12) How do I add a new column to a Table using the Calculator node?
13) How do I read a CSV file from disk?
14) How do I group a set of nodes into a subflow and expose input/output ports?
15) How do I inspect the data flowing on a connection between two nodes?
16) How do I import an Excel file and select a specific sheet?
17) How do I join two tables on a common key using the built-in nodes?
18) How do I filter rows in a table based on a condition (column X $<$ 2000)?
19) How do I convert an ADAF to a Table (and vice versa)?
20) How do I export a plot/figure from a Figure node to a PNG file?
21) How do I package a set of nodes as a node library and install it?
22) How do I pass configuration into a node as data?
23) How can I adapt a python script that I have to run in a X node?

Broader system questions

24) What are the main built-in data types in X, and when would you use each?
25) Can X handle streaming data?

Large workflow questions

26) I want to set up an image analysis pipeline that reads a folder of images, runs a machine learning model for classification, aggregates the results per class, and produces a PDF report — how do I build this step by step?
27) How would I structure an ETL flow that reads from a SQL database, joins with an Excel file, validates rows, and writes the result to both Azure Blob Storage and a PowerBI data source?

Off-topic questions - expected refusals

28) What's the capital of France?
29) Write me a haiku about data engineering.
30) Who is the CEO of Microsoft?
31) What's the recipe for sourdough bread?

Appendix H: Step 4 NFR Measurement Catalogue

TABLE XII: Full NFR Measurement Catalogue

NFR Type	Measurement	Equation	GraphRAG Case Application
Reliability	Uptime (UT)	$UT = (T_{avail}/T_{sched}) \times 100\%$	UT can be used to monitor whether the chatbot service remains reachable during working hours. If UT falls below the defined threshold, the operations team must investigate and restore service availability.
Reliability	Error Rate (ER)	$ER = (N_{fail}/N_{total}) \times 100\%$	ER can be used to detect when the chatbot begins failing requests at an unacceptable rate. If ER rises above the defined threshold, the retrieval or generation pipeline requires debugging.
Usability	Task Completion Rate (TCR)	$TCR = (N_{completed}/N_{attempted}) \times 100\%$	TCR can be used to assess whether users can successfully complete common documentation tasks using the chatbot. Low TCR may indicate unclear responses or missing functionality.
Usability	Perceived Ease of Use (PEoU)	$PEoU = (N_{r\geq 4}/N_{users}) \times 100\%$	PEoU can be used to identify when the chatbot interface or interaction style is perceived as difficult by users. If PEoU falls below the defined threshold, interface or prompt design adjustments are needed.
Explainability	Documentation Reference Rate (DRR)	$DRR = (N_{ref}/N_{rel}) \times 100\%$	DRR can be used to ensure the chatbot consistently provides source references that allow users to verify answers. If DRR falls below the defined threshold, the generation prompt or retrieval context requires adjustment.
Explainability	User-Perceived Understandability (UPU)	$UPU = (N_{r\geq 4}/N_{users}) \times 100\%$	UPU can be used to detect when explanations are unclear or unhelpful to users. Low UPU signals a need to revise explanation templates or generation parameters.
Accuracy	Answer Correctness (AC)	$AC = (N_{correct}/N_{eval}) \times 100\%$	AC can be used to assess the chatbot's reliability in producing correct answers on a representative test set. If AC falls below the defined threshold, retrieval or generation improvements are required.
Accuracy	Groundedness (G)	$G = (N_{grounded}/N_{eval}) \times 100\%$	G can be used to ensure answers stay close to the documentation rather than drifting into general LLM behaviour. If G falls below the defined threshold, retrieval context or generation constraints require tightening.
Accuracy	Hallucination Rate (HR)	$HR = (1/N_{eval}) \times \sum(S_{unsupported}(i)/S_{total}(i)) \times 100\%$	HR can be used to detect when the chatbot produces unsupported or invented content within answers. Elevated HR indicates generation constraints or fact-checking mechanisms need attention.
Performance	Average Response Time (ART)	$ART = (1/N) \times \sum t_i$	ART can be used to monitor whether the chatbot responds quickly enough to keep users engaged. If ART rises above the defined threshold, retrieval or generation optimizations are necessary.
Performance	Maximum Response Time (MRT)	$MRT = \max(t_i)$	MRT can be used to ensure worst-case response times remain within tolerable bounds even for complex queries. If MRT exceeds acceptable levels, query timeout limits or resource allocation require adjustment.
Performance	Throughput (TP)	$TP = N_{queries}/T$	TP can be used to verify the chatbot can handle expected user volumes without queuing delays. Insufficient TP signals the need for infrastructure scaling or load balancing.
Security	Sensitive Data Leakage Count (SDLC)	$SDLC = \sum \mathbf{1}[leak_i]$	SDLC can be used to detect cases where private or confidential information appears in answers or logs. Any non-zero SDLC requires immediate investigation and remediation.
Security	Access Control Compliance (ACC)	$ACC = (N_{authorized}/N_{access}) \times 100\%$	ACC can be used to verify the chatbot only retrieves from authorized data sources. If ACC falls below 100%, access control rules or data source configuration must be corrected.
Scalability	Performance Degradation Under Load (PDUL)	$PDUL = ((ART_{2x} - ART_{1x})/ART_{1x}) \times 100\%$	PDUL can be used to assess whether response time degrades acceptably as user load increases. Excessive PDUL may require caching, load balancing, or infrastructure scaling.

Full NFR Measurement Catalogue (continued)

NFR Type	Measurement	Equation	GraphRAG Case Application
Scalability	Cost Per Query (CPQ)	$CPQ = C_{total}/N_{queries}$	CPQ can be used to monitor whether operational costs remain sustainable as usage grows. If CPQ rises above acceptable levels, model selection, caching strategies, or prompt optimization should be reconsidered.
Maintainability	Documentation Update Time (DUT)	$DUT = t_{searchable} - t_{published}$	DUT can be used to verify new documentation becomes searchable quickly enough to stay synchronized with releases. Elevated DUT indicates the ingestion pipeline needs automation improvements.
Maintainability	Component Replacement Effort (CRE)	$CRE_c = d_c$	CRE can be used to assess whether major components can be replaced within acceptable time limits. High CRE for a component suggests interface abstractions or modularity need improvement.
Maintainability	Mean Time To Repair (MTTR)	$MTTR = (1/N_{issues}) \times \sum(t_{resolved}(i) - t_{reported}(i))$	MTTR can be used to track how quickly issues in the chatbot pipeline are diagnosed and fixed. Elevated MTTR may indicate debugging tools, logging, or system observability need enhancement.
Portability	Deployment Effort (DE)	$DE = (1/N_{deploy}) \times \sum d_i$	DE can be used to verify the chatbot can be deployed to new environments within acceptable time limits. High DE suggests containerisation or deployment automation needs attention.
Portability	Environment Compatibility Count (ECC)	$ECC = \sum \mathbf{1}[pass_i]$	ECC can be used to track how many target environments the chatbot successfully runs in. If ECC falls below the required count, environment-specific dependencies require resolution.
Portability	Setup Dependency Count (SDC)	$SDC = N_{env\text{-}specific}$	SDC can be used to identify environment-specific configuration that hinders portability. Non-zero SDC indicates Docker configuration or dependency management needs revision.
Safety	Unsafe Answer Rate (UAR)	$UAR = (N_{unsafe}/N_{eval}) \times 100\%$	UAR can be used to detect when the chatbot produces harmful or inappropriate recommendations. Elevated UAR signals the need for stronger generation guardrails or content filtering.
Safety	Off-Domain Handling Rate (ODHR)	$ODHR = (N_{handled}/N_{off\text{-}domain}) \times 100\%$	ODHR can be used to verify the chatbot correctly refuses questions outside the intended documentation domain. Low ODHR indicates domain classification or refusal logic requires improvement.
Safety	Prompt Resistance Rate (PRR)	$PRR = (N_{resisted}/N_{attack}) \times 100\%$	PRR can be used to assess the chatbot's resilience to prompt injection and manipulation attempts. Low PRR may require prompt sanitisation or instruction reinforcement.